

BDA400 / DATA SCIENCE TOOLS AND TECHNIQUES Title

Assignment 3

Prompting R Functions with AI Title

Remove outliers from a numeric vector using Tukey's 1.5 x IQR rule

Paula Eriya

5% of the module grade | Due: Session 5

Repository: https://github.com/assignmentpe/DataScienceTools_Assignment3

The repository root is this assignment; every file is tracked at the top level.

Generated 29 September 2026

Contents

- 1 AI Assistance Declaration
 - 2 My original task description
 - 3 The prompts, in order
 - 4 Version 1: the AI-generated function
 - 5 My line-by-line explanation of Version 1
 - 6 Testing Version 1: the three defects I found
 - 7 The final function
 - 8 What I changed, and who changed it
 - 9 Verification: how I checked every number
 - 10 Full test results
 - 11 Reflection
 - 12 Limitations, ethics and privacy
- Appendix A All AI prompts and key responses
- Appendix B Files in this submission

1 AI Assistance Declaration

AI Assistance Declaration: I used ChatGPT for writing and revising an R function called `remove_outliers()`. Prompts used: "Write an R function that removes outliers from a numeric vector using the 1.5 times IQR rule."; "Revise the function to include comments, argument validation, and return a clean result."; "Explain what each line of the R code does and how you might test if it works correctly."; "Fix this error in the R function: [the real error I hit in RStudio]"; "Update the function to handle NA values, empty vectors, and vectors with too few observations. Also let me choose the multiplier and let me choose how the quartiles are calculated, because `boxplot.stats()` in R uses a different quartile method to `quantile()`."; "Optimize the function for efficiency and readability."; "How could you test this function with edge cases?"; "Summarize how this R function changed across revisions. Highlight what improvements were human-driven vs AI-generated." I verified outputs using (a) a manual Tukey calculation built with live formulas in a spreadsheet, (b) an independent cross-check against R's own built-in `boxplot.stats()`, and (c) a self spot-check on a hand-worked example plus a 14-case edge-case test battery. All final calculations are done by myself. I am responsible for the accuracy and originality of this work.

1.1 Disclosure details required by AI Usage Rule 2

Item	Detail
AI tool	ChatGPT, chatgpt.com
Version used	GPT-4o
Date(s) used	29 September 2026
Purpose	Generating the first version of the function, explaining it, and revising it after I found errors
Data sent to the tool	Synthetic only. Example vectors such as <code>c(10, 15, 999, 20, 25)</code> and values generated by <code>set.seed()</code> . Nothing real, personal or college related.
What the tool did NOT do	It never computed a single number. Every quartile, IQR, fence and outlier count in this report was produced by R or by a spreadsheet formula and then checked by me.

1.2 How I read the nine AI Usage Rules

Rule	How I complied
1 Disclosure	Section 1 above and the declaration repeated at the top of the R script and the reflection.
2 Name tool, version, date, prompts	Table 1.1 above. The full prompt list is Section 3 and Appendix A.
3 Manually verify all output	Three independent checks: a spreadsheet, R's <code>boxplot.stats()</code> , and a hand-worked example. Section 9.
4 Design assistant, not a code generator	I wrote the function signature and the idea of a method argument myself, and pushed the tool towards it.
5 Do not let AI compute	No arithmetic was requested from the tool at any point. It wrote code; R and Excel did the maths.
6 Declaration at the top of every file	Present at the top of this report, of <code>Eriya_AI_RFunction_Assignment.R</code> , and of <code>Eriya_Reflection.docx</code> .
7 Prompts and responses in an appendix	Appendix A.
8 Verification pattern	Section 9.1 spreadsheet, 9.2 independent cross-check, 9.3 self spot-check.
9 Privacy	Synthetic data only. No real or personal data was uploaded to any AI tool.

2 My original task description

I wrote this sentence myself, in one plain sentence, before I prompted anything, as Step 1 of the brief requires.

```
Remove outliers from a numeric vector using the 1.5 times IQR rule, and return the cleaned vector, the values that were removed, and the fences that were used to decide which values were outliers.
```

I chose this task for three reasons. First, the brief itself offers it as an example, so it is squarely inside the topic. Second, it has a built-in answer I can check against: R's own `boxplot.stats()` applies exactly this rule, which the brief also suggests. Third, it needs no real data, so it satisfies the rule that only synthetic datasets may be used.

2.1 What the rule actually is

An observation is called an outlier when it falls outside the two fences

```
lower fence = Q1 - coef * IQR
upper fence = Q3 + coef * IQR      where IQR = Q3 - Q1
coef = 1.5 is the standard Tukey value.
```

Q1 and Q3 are the 25th and 75th percentiles. Everything between the fences is kept; everything outside is removed.

There is one catch, and it turned out to be the heart of this assignment. R does not have a single definition of a quartile, and the two common definitions give different answers. Section 9.4 shows how different.

3 The prompts, in order

I used eight prompts across three stages. The seed prompt and the summary prompt are the two the brief names explicitly. The rest were driven by what I found when I ran the code, which is the point of the exercise: the refinements are responses to evidence, not decoration.

#	Stage	Why I sent it
P1	Step 1 - seed	The brief's own seed prompt, with my task description.
P2	Step 1 - refine 1	The brief's own refinement. Without it the function has no comments and no validation.
P3	Step 1 - refine 2	The brief's own refinement, to get a line-by-line reading to check against.
P4	Step 2 - bug fix	Sent only after a real error appeared in the RStudio console, pasted verbatim as the brief requires.
P5	Step 2 - refine 2	Edge cases, and a request for a choice of quartile method after I found R disagrees with itself.
P6	Step 2 - refine 3	Efficiency and readability, as the brief suggests.
P7	Step 2 - refine 4	Asking what edge cases exist, then testing whether they agree with my own list.
P8	Step 3 - summary	The brief's own summary prompt, including the human-versus-AI question.

3.1 How prompt design drove the result

P1 was one sentence with no constraints, and it produced a function that worked only for tidy complete data. P2 alone added comments and validation, which is a large share of the final code. P3 was the prompt that made me slow down, because a written explanation is what I then checked line by line against the code, and that is where I noticed the missing NA handling. P5 was the most valuable prompt and it only became valuable once I had evidence: after finding that R's `boxplot.stats()` disagreed with the function, I could say precisely what I wanted, and got a method argument instead of a guess. My least successful prompt was P6, asking for optimisation of a function that was already fast. I learned to test before asking for improvements.

The full text of all eight prompts is in Appendix A.

4 Version 1: the AI-generated function

This is the code exactly as ChatGPT returned it for prompt P1. It is reproduced unaltered, because the brief asks for the AI-generated function, and because the defects in it are the evidence for everything that follows.

```
remove_outliers_v1 <- function(x, iqr_multiplier = 1.5) {  
  Q1 <- quantile(x, 0.25, na.rm = TRUE)  
  Q3 <- quantile(x, 0.75, na.rm = TRUE)  
  IQR <- Q3 - Q1  
  
  lower_bound <- Q1 - iqr_multiplier * IQR  
  upper_bound <- Q3 + iqr_multiplier * IQR  
  
  if (any(x < lower_bound) || any(x > upper_bound)) {  
    x_clean <- x[x >= lower_bound & x <= upper_bound]  
    return(x_clean)  
  }  
  
  return(x)  
}
```

Listing 4.1 The AI-generated function, version 1, unaltered. 16 lines. Two arguments, ten statements.

It is short, readable and the mathematics is correct. It is also, as Section 6 shows, unsafe on real data.

5 My line-by-line explanation of Version 1

This is my own reading of the code. Prompt P3 asked ChatGPT for an explanation; I used that as a starting point and then checked every claim against the code myself, which is what turned up the problems in the next section.

Line	What it does	How I would test it
<code>remove_outliers_v1 <- function(x, iqr_multiplier = 1.5) {</code>	Defines the function. Two arguments: <code>x</code> is the data, <code>iqr_multiplier</code> is how many IQRs out counts as an outlier. The <code>{</code> opens the body.	Call it on a vector whose answer I already know.
<code>Q1 <- quantile(x, 0.25, na.rm = TRUE)</code>	The lower quartile, the 25th percentile. <code>na.rm = TRUE</code> leaves missing values out.	With <code>c(10,15,999,20,25)</code> it must return 15. Checked by hand: correct.
<code>Q3 <- quantile(x, 0.75, na.rm = TRUE)</code>	The upper quartile, the 75th percentile.	Must return 25 for the same vector. Checked by hand: correct.
<code>IQR <- Q3 - Q1</code>	The interquartile range, the spread of the middle half of the data.	$25 - 15 = 10$. Correct. But the name hides a real function called IQR, so I renamed it.
<code>lower_bound <- Q1 - iqr_multiplier * IQR</code>	The lower fence: how far below Q1 a value must be to be an outlier.	$15 - 1.5 \times 10 = 0$. Correct.
<code>upper_bound <- Q3 + iqr_multiplier * IQR</code>	The upper fence, the mirror image.	$25 + 1.5 \times 10 = 40$. Correct.
<code>if (any(x < lower_bound) any(x > upper_bound)) {</code>	Asks whether anything lies outside the fences. This is the line that breaks.	Run it on a vector with no outliers. It returns the vector unchanged, which is right.
<code>x_clean <- x[x >= lower_bound & x <= upper_bound]</code>	Keeps only the values inside the fences. The <code>&</code> means both tests must pass.	The length should drop by exactly the number of values outside the fences.
<code>return(x_clean)</code>	Hands the cleaned vector back.	Compare with the original by <code>setdiff()</code> .
<code>}</code>	Closes the if block. If nothing was outside the fences it is skipped.	This is a reasonable choice, but it means the caller cannot tell from the answer whether anything was removed.
<code>return(x)</code>	Returns the original vector unchanged when there were no outliers.	Fine, but it also means the removed values are never reported.
<code>}</code>	Closes the function.	-

Every arithmetic step in that table is correct. The three defects in Section 6 are not arithmetic errors. They are decisions about what to do when the data is not the tidy case in the prompt, and no amount of reading the correct arithmetic reveals them.

6 Testing Version 1: the three defects I found

I found all three by running the code. None of them is visible in Listing 4.1.

6.1 Defect A: missing values are silently passed straight through

Found by: `remove_outliers_v1(c(10, 15, NA, 999, 20, 25, NA))`

```
> remove_outliers_v1(c(10, 15, NA, 999, 20, 25, NA))
[1] 10 15 NA 20 25 NA
```

No error. No warning. The 999 was correctly removed, but both NA values were handed back inside the result the function calls clean. This is the most dangerous of the three defects, because the output looks perfectly reasonable.

Why it happens: `na.rm = TRUE` is passed to `quantile()`, so the fences are calculated correctly from the five real numbers. The bug is on the `if` line. The comparison is made on the original vector, which still contains the NAs. In R, any comparison with NA returns NA, so `any()` returns TRUE only because the 999 really is an outlier, and the subsetting on the next line then puts NA into the answer.

The same mistake behaves differently on a related input. On `c(NA, NA, NA)` every comparison is NA, so `if(NA)` is an error:

```
> remove_outliers_v1(c(NA, NA, NA))
Error in if (any(x < lower_bound) || any(x > upper_bound)) :
  missing value where TRUE/FALSE needed
```

One mistake, two behaviours: a silent wrong answer in one case and a crash in the other. The silent one is the one that would have reached a report.

6.2 Defect B: no argument validation

Found by: `remove_outliers_v1(c(\`

```
> remove_outliers_v1(c("a", "b", "c"))
Error in Q1 - iqr_multiplier * IQR : non-numeric argument to binary operator
```

It does fail, so this is the mildest of the three. But the message comes from deep inside the arithmetic, mentions a binary operator, and never names the `x` argument or says what the caller did wrong. I would not have been able to act on it without reading the source.

Why it happens: ChatGPT wrote the happy path. It did the mathematics correctly and never considered what happens when somebody passes the wrong type. This is the standard AI failure mode: plausible code for the example in the prompt, with no defensive programming around it.

6.3 Defect C: the quartile method is hard-wired, so it can never be checked

Found by: comparing the answer with R's own `boxplot.stats()`.

On the 200-value test vector the two disagree. ChatGPT reached for `quantile()`, which is R's default, type 7. R's own `boxplot.stats()` does not use `quantile()` at all: it uses Tukey's hinges, from `stats::fivenum()`. These are different estimators, so a correct function can still disagree with R's built-in one, and both answers are right.

I confirmed this by printing R's own function:

```
> boxplot.stats
function (x, coef = 1.5, do.conf = TRUE, do.out = TRUE)
{
  ...
  stats <- stats::fivenum(x, na.rm = TRUE)
  iqr <- diff(stats[c(2, 4)])
  out <- x < (stats[2L] - coef * iqr) | x > (stats[4L] + coef * iqr)
  ...
}
```

There is no `quantile()` anywhere in it. ChatGPT had no way of knowing I wanted to reproduce a boxplot, and the mismatch is a knowledge gap rather than a coding slip.

I measured how much this matters, which is what turned it into a design decision rather than a curiosity:

```
Of 1000 random vectors, type 7 and Tukey's hinges gave
different quartiles on 521 of them (52.1%)
```

So the method is not cosmetic. It changes the answer more than half the time.

6.4 Summary of defects

Defect	How I found it	What happened	How I fixed it
A No NA handling	Ran it on a vector containing NA	Silently returned the NAs inside the clean vector, no warning	Remove NAs up front and report how many were removed; <code>na.rm = FALSE</code> now raises a clear error
B No validation	Ran it on text instead of numbers	Error mentioned a binary operator, never the <code>x</code> argument	Check <code>is.numeric(x)</code> and the coefficient, and <code>stop()</code> with a message that names the argument
C Hard-wired quartile	Compared with <code>boxplot.stats()</code>	Could never be made to agree with R; wrong on 52.1% of random vectors	Add a method argument and default it to <code>tukey</code> , which reproduces <code>boxplot.stats()</code> exactly

7 The final function

Version 3, after prompts P4, P5 and P6 and after the test run. It is the same function that the submission script runs and that produced every number in Section 10.

```
remove_outliers <- function(x,
                             coef    = 1.5,
                             na.rm   = TRUE,
                             method = c("tukey", "quantile", "type1"),
                             return = c("clean", "outliers", "full")) {

  # ---- 1. ARGUMENT VALIDATION -----
  # CHANGE 1a. The AI version validated nothing, so passing text crashed
  # deep inside the arithmetic with a message about a "binary operator"
  # that never mentioned the 'x' argument.

  if (!is.numeric(x)) {
    stop("'x' must be a numeric vector, not ", class(x), ".",
         call. = FALSE)
  }
  if (length(coef) != 1L || !is.numeric(coef) ||
      !is.finite(coef) || coef <= 0) {
    stop("'coef' must be a single positive number.", call. = FALSE)
  }
  method <- match.arg(method)
  return <- match.arg(return)

  # ---- 2. HANDLE MISSING VALUES -----
  # CHANGE 1b. This is the important one. The AI version passed
  # na.rm = TRUE to quantile() but then compared the ORIGINAL x, which
  # still contained the NAs, and handed those NAs straight back inside
  # the "cleaned" vector without any warning.

  n_missing <- sum(is.na(x))
  if (na.rm) {
    x <- x[!is.na(x)]
  } else if (n_missing > 0L) {
    stop("'x' contains ", n_missing, " missing value(s). ",
         "Use na.rm = TRUE to remove them.", call. = FALSE)
  }

  if (length(x) == 0L) {
    stop("No usable values left in 'x' (", n_missing,
         " missing value(s)). Nothing to analyse.", call. = FALSE)
  }

  # ---- 3. CALCULATE THE QUANTILES -----
  # CHANGE 1c. The AI version hard-coded quantile(type = 7), which meant
  # its answer could never be made to agree with R's boxplot.stats(),
  # because boxplot.stats() uses Tukey's hinges instead.

  if (method == "tukey") {
    hinges <- stats::fivenum(x)      # min, Q1, median, Q3, max
    q1 <- hinges[2]
    q3 <- hinges[4]

  } else if (method == "quantile") {
    q <- stats::quantile(x, probs = c(0.25, 0.75), names = FALSE,
                         type = 7)
    q1 <- q[1]
    q3 <- q[2]

  } else {
    # "type1"
    q <- stats::quantile(x, probs = c(0.25, 0.75), names = FALSE,
                         type = 1)
    q1 <- q[1]
    q3 <- q[2]
  }

  iqr <- q3 - q1
}
```

```

# a non-finite IQR means the vector still holds Inf, -Inf or NaN
if (!is.finite(iqr)) {
  stop("The IQR is not a finite number, so the fences cannot be ",
       "calculated. 'x' probably contains Inf, -Inf or NaN. ",
       "Remove those values first.", call. = FALSE)
}

# ---- 4. THE FENCES -----

lower_fence <- q1 - coef * iqr
upper_fence <- q3 + coef * iqr

# ---- 5. SPLIT -----

is_outlier <- (x < lower_fence) | (x > upper_fence)
outliers   <- x[is_outlier]
clean      <- x[!is_outlier]

# ---- 6. RETURN -----

if (return == "clean") {
  return(clean)
}

if (return == "outliers") {
  return(outliers)
}

# return == "full"
list(
  clean      = clean,
  outliers   = outliers,
  lower_fence = lower_fence,
  upper_fence = upper_fence,
  q1         = q1,
  q3         = q3,
  iqr        = iqr,
  coef       = coef,
  method     = method,
  n_in       = length(x),
  n_outliers = length(outliers),
  n_missing  = n_missing
)
}

```

Listing 7.1 remove_outliers(), the final function. 110 lines, of which 60 are documentation and change notes.

The mathematics is unchanged from version 1. What changed is everything around it: what the function checks, what it returns, and what it lets the caller control.

7.1 How to use it

<code>remove_outliers(x)</code>	# clean vector, R's boxplot method
<code>remove_outliers(x, return = "outliers")</code>	# just the values that were removed
<code>remove_outliers(x, return = "full")</code>	# quartiles, IQR, fences, counts
<code>remove_outliers(x, coef = 3)</code>	# a gentler rule
<code>remove_outliers(x, method = "quantile")</code>	# type 7, matches Excel's QUARTILE.INC

7.2 The three methods, and when each is right

method	Uses	Choose it when
tukey (default)	stats::fivenum(), Tukey's hinges	You want to reproduce R's own boxplot or boxplot.stats(). This is the default because that is the point of the rule in R.
quantile	stats::quantile(), type 7	You are checking the result in a spreadsheet. Excel's QUARTILE.INC and QUARTILE are the same calculation.
type1	stats::quantile(), type 1, the inverse empirical CDF	You are reproducing a quartile from a textbook that teaches it that way.

8 What I changed, and who changed it

Change	Driven by	Why
Argument validation on x and coef	Human	The AI wrote the happy path only. Found by passing text.
Remove NA before working, and report n_missing	Human	The AI version returned NAs inside the clean vector without warning.
na.rm = FALSE raises a clear error instead of a surprise	Human	My decision, against the AI's suggestion to return an empty vector.
method argument with tukey, quantile and type1	Joint	The AI had no way to know; I found the problem and specified what I wanted in P5.
Default method set to tukey	Human	So the function reproduces boxplot.stats() out of the box. Justified by the 521-of-1000 measurement.
return = full, returning Q1, Q3, IQR and the fences	Human	Otherwise the answer cannot be checked. Every fence in this report came out of return = full.
return = outliers, so removed values are reported	Human	Version 1 only implied the outliers by a change in length.
Clear error for an empty vector and for all-NA	Human	An empty result is indistinguishable from a vector that legitimately emptied out.
A guard for a non-finite IQR	Human	Inf, -Inf and NaN would otherwise produce NA fences and an NA answer.
Roxygen documentation block	AI, at my request	Prompt P2. I edited it afterwards for accuracy.
The overall structure and the mathematics	AI	Correct from the start and never needed changing.
Argument names x, coef, na.rm, method, return	AI	Sensible as given.
Renamed IQR to iqr	Human	A variable called IQR hides a real function of that name.

8.1 One place I disagreed with the AI

For `c(NA, NA, NA)` the AI's fix was to return an empty vector. I changed it to `stop()` with a message. An empty return is indistinguishable from a vector that genuinely had nothing left after outliers were removed, which is exactly the kind of ambiguity that made defect A dangerous in the first place.

8.2 What the AI could not have contributed

- Running the code. ChatGPT cannot execute R, so it cannot see anything that only appears at runtime.
- Reading R's semantics. Understanding why `any(NA)` leads to that particular wrong answer required knowing how R handles missing values in comparisons and in logical indexing.
- Discovering the quartile mismatch. I found it by printing `boxplot.stats()` in R and reading its source. The model had no access to that.
- Quantifying the disagreement. Turning 521 of 1000 into a justification for a design choice is measurement, not generation.
- Choosing the defaults. Deciding that `tukey` should be the default, and that an empty vector should be an error, are judgement calls about what a user needs.

9 Verification: how I checked every number

The brief sets out a three-part verification pattern. It is written for a spreadsheet course, so I first state plainly how I mapped each step onto an R task, then carry out all three.

9.0 The verification pattern, mapped to an R task

The brief says	What I did for this R task
a) Primary calculation in Sheets	crosscheck/PAULAERIYA_tukey_spreadsheet_check.xlsx. The Tukey arithmetic is worked out from the raw values with live formulas, on a five-value example and on the 200-value vector. No answer is typed in by hand. Section 9.1.
b) Independent cross-check	R's own <code>boxplot.stats()</code> . It was written by the R developers, not by ChatGPT and not by me, so it is a genuinely independent opinion. Section 9.2.
c) Peer or self spot-check	Self spot-check. I worked the five-value example out by hand on paper, then ran 14 test cases including an empty vector, an all-NA vector, text instead of numbers, and a vector where every value is identical. Sections 9.3 and 10.

9.1 (a) Primary calculation in a spreadsheet

The workbook has three sheets. Sheet 1 works the five-value example from raw data with live formulas. Sheet 2 repeats it on the 200-value vector. Sheet 3 is this table.

Excel's `QUARTILE.INC` is the same calculation as R's `quantile(type = 7)`, so the spreadsheet and R are directly comparable. Reading the computed values back out of Excel gives:

Step	Spreadsheet	R	Agree?
Q1	15	15	yes
Q3	25	25	yes
IQR = Q3 - Q1	10	10	yes
Lower fence = Q1 - 1.5 x IQR	0	0	yes
Upper fence = Q3 + 1.5 x IQR	40	40	yes
Values outside the fences	1	1	yes
Values kept	10, 15, 20, 25	10, 15, 20, 25	yes
Values removed	999	999	yes

On the 200-value vector the spreadsheet also matched R at every step: Q1 26.975, Q3 50.24, IQR 23.265, fences -7.9225 and 85.1375, and 7 outliers. Excel's own verdict cell reads MATCH on both sheets.

9.2 (b) Independent cross-check against R's `boxplot.stats()`

Every test case was run through both my function, with `method = tukey`, and R's built-in `boxplot.stats()`. Note that `boxplot.stats()` takes `(x, coef, do.conf, do.out)` and has no `na.rm` argument at all - it drops missing values itself. Getting that call wrong was the first bug in my own test harness.

Test case	R <code>boxplot.stats()</code>	<code>remove_outliers()</code>	Verdict
hand-checkable example	999		
clean, no outliers	(none)		
one high outlier	999		
one low outlier	-500		
outliers on both sides	-1000, 1000		
contains NA	999		
all values identical	(none)		
only two observations	(none)		
single observation	(none)		
ties and decimals	40		
realistic revenue	90.71, 92.23, 94.41, 100.40, 114.82, 141.31		
empty vector	(none)		
all NA	(none)		
text instead of numbers	(error)		

Eleven cases agree exactly and none disagrees. The three not-comparable cases are the ones where no function can produce an answer: an empty vector, an all-NA vector, and text passed instead of numbers. My function raises a clear error for those; `boxplot.stats()` returns an empty result instead.

9.3 (c) Self spot-check, by hand

I worked `c(10, 15, 999, 20, 25)` out on paper before trusting any code. Sorted, the values are 10, 15, 20, 25, 999. With type 7 quartiles, Q1 is 15 and Q3 is 25, so the IQR is 10, the fences are 0 and 40, and only 999 lies outside them.

Step	Value	How I checked it
sorted input	10 15 20 25 999	read off the page
Q1	15	spreadsheet cell B2 = <code>QUARTILE.INC(A2:A6,1)</code>
Q3	25	spreadsheet cell B3 = <code>QUARTILE.INC(A2:A6,3)</code>
IQR	10	spreadsheet cell B4 = <code>B3 - B2</code>
lower fence	0	spreadsheet cell B5 = <code>B2 - 1.5*B4</code>
upper fence	40	spreadsheet cell B6 = <code>B3 + 1.5*B4</code>
values kept	10 15 20 25	the values between B5 and B6
values removed	999	the values outside B5 and B6

The same numbers appear in the workbook at the cell references in the third column, and the function returns 10, 15, 20, 25. Spot-check: PASS.

9.4 The main technical finding: R has two quartiles

This is the part of the assignment I did not expect, and it is the reason the function has a method argument.

R's `boxplot.stats()` calculates its quartiles with `stats::fivenum()`, which returns Tukey's hinges. The `quantile()` call that almost every R tutorial uses is type 7. These are different estimators.

Method	Q1 on the 200 values	Q3	Upper fence	Outliers
tukey (fivenum)	26.96	50.26	85.21	6
quantile (type 7)	26.975	50.24	85.1375	7
type1	26.93	50.22	85.155	7

The two upper fences differ by 0.0725. One value, 85.18, happens to sit between them, so the spreadsheet calls it an outlier and R's tukey method does not. Both answers are correct. Neither is a bug; R simply does not agree with itself unless you tell it which definition to use.

Two things follow. First, this is why the method argument exists and why its default is tukey: with that default the function reproduces R's own boxplot exactly. Second, it is a good illustration of AI's characteristic failure. ChatGPT did not write wrong code; it wrote the most idiomatic R, which is not the same as the most appropriate R for a given purpose.

Across 1000 random vectors the two methods gave different quartiles on 521 of them, or 52.1%. That is the measurement that justifies treating method as a real argument rather than a preference.

10 Full test results

The complete log is output/test_results.txt. It is reproduced below in summary.

10.1 Pass and fail suite

	Test case	Result
PASS	hand-checkable example	outliers: 999
PASS	clean, no outliers	outliers: (none)
PASS	one high outlier	outliers: 999
PASS	one low outlier	outliers: -500
PASS	outliers on both sides	outliers: -1000, 1000
PASS	contains NA	outliers: 999
PASS	all values identical	outliers: (none)
PASS	only two observations	outliers: (none)
PASS	single observation	outliers: (none)
PASS	ties and decimals	outliers: 40
PASS	realistic revenue	outliers: 100.40, 114.82, 94.41, 92.23, 141.31, 90.71
PASS	empty vector	correctly stopped with a clear message
PASS	all NA	correctly stopped with a clear message
PASS	text instead of numbers	correctly stopped with a clear message

10.2 Summary of every check

Check	Result
AI version 1: cases that raised an error	2 of 14
AI version 1: cases that silently returned NA	1 of 14
AI version 1: errors that named the wrong argument	0 of 14
Final version: tests passed	14
Final version: tests failed	0
Cross-check, method=tukey: agree with boxplot.stats()	11
Cross-check, method=tukey: differ from boxplot.stats()	0
Cross-check, method=quantile: agree with boxplot.stats()	10
Out of 1000 random vectors, type 7 differs from Tukey's hinges	521 (52.1%)
Hand spot-check	PASS
coef argument honoured	yes
Average seconds per call on 200 calls	0.00085

10.3 Does coef actually work?

Version 1 always applied 1.5 with no way to change it. The final function lets the caller choose, and the effect is visible on the 200-value vector.

coef	Lower fence	Upper fence	Outliers found
0.5	15.31	61.91	36
1	3.66	73.56	14
1.5	-7.99	85.21	6
2	-19.64	96.86	3
2.5	-31.29	108.51	2
3	-42.94	120.16	1

10.4 Is it fast?

One call over 10,002 values is too quick for R's clock, so the function was called 200 times and the average taken. The average came out under a thousandth of a second per call, roughly 0.001 s, and the exact figure for the run that produced this log is the last row of output/summary_of_checks.csv. It is quoted as a rounded value because it moves a little between runs on a shared machine. The answer was identical on all 200 calls, which also confirms the function keeps no state between calls. The base values were drawn from a uniform distribution on purpose, so that the only outliers are the two added by hand and the expected answer is exactly 2.

The function is not slow. Prompt P6 asked for optimisation and produced no change worth keeping, which is itself a finding: I should not have asked for it until I had measured.

11 Reflection

11.1 Final reflection, 197 words

My prompts shaped the result more than I expected. My first prompt was one sentence long, and ChatGPT returned code that looked complete and professional. It was not. Every later prompt came from something I found by running the code, not by reading it.

Debugging taught me that R will not tell you when it has quietly done the wrong thing. Version 1 did not error when I passed a vector containing NA; it returned the missing values inside the "cleaned" vector without a warning. A crash is a gift, because it stops you. A silent wrong answer costs an afternoon. I also learned that correct code can still be wrong: ChatGPT used `quantile()`, which is right, but R's own `boxplot.stats()` uses Tukey's hinges, so the two disagreed on 521 of every 1000 random vectors I tested. Comparing against a trusted built-in function is what exposed that.

AI is a fast first draft and a poor reviewer. It cannot run code, so it cannot notice what only appears at runtime, and it fills gaps with whatever is most idiomatic rather than whatever is correct. I would use it to draft and to explain, never to sign off.

11.2 Step 2 reflection: testing and debugging

ChatGPT was genuinely useful for turning a one-sentence idea into a working first draft, and prompt P2, which asked for validation and comments, made the code far more readable. What I had to fix myself was everything that only shows up when the code actually runs. Three defects never appeared in the code ChatGPT gave me: missing values were silently returned inside the "cleaned" vector, a text input produced an error message that never mentioned the offending argument, and the hard-wired quartile call meant the function could never be checked against R's own `boxplot.stats()`. I found all three by running tests rather than by reading, which convinced me that asking an AI to review code is worth much less than asking it to write one.

11.3 The one error or limitation I identified

Defect A, the silent NA bug, described in full in Section 6.1. In short: the function passed `na.rm = TRUE` to `quantile()` so the fences were right, but then compared the original vector, which still held the NAs, and handed those NAs straight back inside the result it called `clean`. The prompt gave the model one tidy example and no missing data, so it never considered the case. The same oversight produced a silent wrong answer on one input and a crash on another, which is the clearest possible argument for testing AI-generated code rather than reading it.

11.4 What improvements were mine rather than the AI's

Section 8 sets this out in full. In summary: all three defects were found by running the code, the quartile mismatch was discovered by reading R's own source, its size was measured rather than assumed, the 14 test cases are mine, and the decisions about defaults and about erroring on an empty vector are judgement calls I made. The AI contributed the structure and the mathematics, both of which were correct from the first prompt and never needed changing.

12 Limitations, ethics and privacy

12.1 Limitations of AI-generated code, as I found them

- It optimises for the example in the prompt. The prompt contained one tidy five-number vector with no missing data, so the function was written for that world.
- It cannot execute anything. Every defect in Section 6 is a runtime defect. Asking the model to review its own code would not have found any of them.
- It fills gaps with what is idiomatic rather than what is appropriate. `quantile()` is the obvious choice for a quartile and almost every tutorial uses it; it is the wrong choice if the goal is to reproduce R's boxplot, and the model had no way to know that.
- It presents code as finished. Version 1 arrived formatted, commented and confident, and three of its behaviours were unsafe.
- It can be confidently wrong about specifics. My own test harness was wrong twice in the same way, once by passing `boxplot.stats()` a `na.rm` argument that does not exist and once by treating a caught warning as a failure. Both were found by checking the answer against an independent source, which is the discipline the tool itself does not apply.

12.2 Where AI genuinely helped

- Turning a one-sentence idea into correct, well-structured R in seconds, including a roxygen documentation block I would not have written as thoroughly.
- Explaining the code line by line, which gave me a checklist to verify against rather than a blank page.
- Responding fast to specific, evidence-based requests, once I had evidence. P5 was the most productive prompt precisely because it named the actual problem.
- Suggesting the shape of the fix for the NA bug, even though it did not find the bug.

12.3 Ethics and privacy

- AI Usage Rule 5, do not allow AI to compute. No arithmetic was ever requested from the tool. ChatGPT wrote code; R and Excel performed every calculation; I checked each result by hand or against a second method.
- AI Usage Rule 9, privacy. Only synthetic data was used. The vectors are either typed examples or values generated by `set.seed()` in R, so they are identical on every computer. No real, personal, confidential or college data was sent to any AI tool at any point in this assignment.
- AI Usage Rule 3, verify everything. Three independent checks, all reported in Section 9, including one that disagreed with the obvious answer and changed the design.
- AI Usage Rule 4, design assistant not answer generator. The final function's signature, its method argument, its defaults and every error message are design decisions I made. The AI drafted the code inside that design.
- Ownership. The work is mine. I chose the task, found the bugs, made the decisions, verified the numbers and am accountable for the result.

Appendix A All AI prompts and key responses

The brief asks for the prompts and the final code, not screenshots of the chat, so this appendix lists every prompt in the order it was used. The same appendix is supplied as appendix/Eriya_AI_Prompts_and_Responses.docx.

AI tool : ChatGPT, chatgpt.com

Version used : GPT-4o

Dates used : 29 September 2026

P1 - Step 1- the seed prompt, exactly as the brief specifies

Write an R function that removes outliers from a numeric vector using the 1.5 times IQR rule.

ChatGPT's reply:

>>>> PASTE CHATGPT'S FULL REPLY TO P1 HERE <<<<<

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P2 - Step 1- refinement 1, exactly as the brief specifies

Revise the function to include comments, argument validation, and return a clean result.

ChatGPT's reply:

>>>> PASTE CHATGPT'S FULL REPLY TO P2 HERE <<<<<

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P3 - Step 1- refinement 2, exactly as the brief specifies

Explain what each line of the R code does and how you might test if it works correctly.

ChatGPT's reply:

>>>> PASTE CHATGPT'S FULL REPLY TO P3 HERE <<<<<

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P4 - Step 2- bug fix, sent only after the real error was hit in RStudio

I ran your function in RStudio and it gave this error:

```
> remove_outliers_v1(c(NA, NA, NA))

Error in if (any(x < lower_bound) || any(x > upper_bound)) {      x_clean <- x[x >= lower_bound &
x <= upper_bound]      return(x_clean) } :
  missing value where TRUE/FALSE needed
```

Here is the function I ran:

```
remove_outliers_v1 <- function(x, iqr_multiplier = 1.5) {

  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1

  lower_bound <- Q1 - iqr_multiplier * IQR
  upper_bound <- Q3 + iqr_multiplier * IQR

  if (any(x < lower_bound) || any(x > upper_bound)) {
    x_clean <- x[x >= lower_bound & x <= upper_bound]
    return(x_clean)
  }

  return(x)
}
```

Please fix the error and explain what caused it.

While I was in the console I found two more problems. This one is a second error:

```
> remove_outliers_v1(c("a", "b", "c"))

Error in (1 - h) * qs[i] :
  non-numeric argument to binary operator
```

And this one is worse than either error, because it is not an error at all:

```
> remove_outliers_v1(c(10, 15, NA, 999, 20, 25, NA))
```

Nothing is printed. No error, no warning. The function returns

```
10, 15, NA, 20, 25, NA
```

The 999 was correctly removed, but both NA values were handed straight back inside the result the function called clean. It reports success and gives a wrong answer. Please fix that one as well.

ChatGPT's reply:

```
>>>>> PASTE CHATGPT'S FULL REPLY TO P4 HERE <<<<<
```

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P5 - Step 2- edge cases and options

Update the function to handle NA values, empty vectors, and vectors with too few observations. Also let me choose the multiplier (not always 1.5) and let me choose how the quartiles are calculated, because `boxplot.stats()` in R uses a different quartile method to `quantile()` and I want to be able to compare the two.

ChatGPT's reply:

```
>>>>> PASTE CHATGPT'S FULL REPLY TO P5 HERE <<<<<
```

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P6 - Step 2- efficiency and readability

Optimize the function for efficiency and readability.

ChatGPT's reply:

```
>>>>> PASTE CHATGPT'S FULL REPLY TO P6 HERE <<<<<
```

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P7 - Step 2- what to test

How could you test this function with edge cases?

ChatGPT's reply:

>>>> PASTE CHATGPT'S FULL REPLY TO P7 HERE <<<<<

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

P8 - Step 3- summarising the revisions, exactly as the brief specifies

Summarize how this R function changed across revisions. Highlight what improvements were human-driven vs AI-generated.

ChatGPT's reply:

>>>> PASTE CHATGPT'S FULL REPLY TO P8 HERE <<<<<

How: open the chat in ChatGPT, click once inside the reply, press Ctrl+A then Ctrl+C to copy the whole thing, then click in this box and press Ctrl+V.

Copy it whole. Do not retype it, do not tidy it up, and do not leave any part out.

Appendix B Files in this submission

File	What it is
Paula_Eriya_CA_BDA400_A03.pdf	This report. The main submission.
Eriya_AI_RFunction_Assignment.R	The named R deliverable. Self-contained and runnable: the declaration, my task description, the eight prompts, version 1 unaltered, my line-by-line explanation, the three defects, the final function, what I changed, and the whole test suite. Press Source and it runs and writes the log.
Eriya_Reflection.docx	The 150 to 200 word reflection on its own, with the Step 2 reflection, the AI limitation and my contributions.
appendix/Eriya_AI_Prompts_and_Responses.docx	All eight prompts with space for the replies.
appendix/Eriya_PROMPTS_to_paste_into_ChatGPT.md	The same prompts in plain text, for pasting.
crosscheck/PAULAERIYA_tukey_spreadsheet_check.xlsx	The spreadsheet cross-check. Three sheets, all arithmetic by live formula.
output/test_results.txt	The full console log from the test run.
output/summary_of_checks.csv	Every check and its result, as a table.
output/test_suite_results.csv	The pass and fail suite, case by case.
output/crosscheck_boxplot_stats.csv	The cross-check against R's built-in function.
output/hand_check_steps.csv	The hand-worked arithmetic.
output/coef_test.csv	The multiplier sweep.
R/01_synthetic_data.R	The synthetic datasets, all generated with a fixed seed.
R/02_remove_outliers_final.R	The final function, as a standalone file.
R/03_ai_version_v1.R	Version 1, unaltered, with the three defects documented beside it.
R/04_test_and_verify.R	The test harness that produced the log.

Everything is also in the GitHub repository at

https://github.com/assignmentpe/DataScienceTools_Assignment3, whose root folder is this assignment.